

Nearest Neighbours: KD-Tree vs Quad-Tree

Project Proposal

1 Nearest Neighbours: KD-Tree vs Quad-Tree

1.0.1 Project Proposal

1. Problem Formulation

The **Nearest Neighbour Search (NNS)** problem is defined as follows: given a dataset $P = \{p_1, p_2, \dots, p_n\} \subset \mathbb{R}^m$ and a query point $q \in \mathbb{R}^m$, find the k points in P that minimise the Euclidean (L2) distance to q :

$$\text{KNN}(q, k) = \arg \min_{S \subseteq P, |S|=k} \sum_{p \in S} \|q - p\|_2$$

NNS is a foundational primitive in robotics, autonomous vehicles (point cloud registration, obstacle detection), computer vision, and machine learning. As dataset sizes scale, naive exhaustive search becomes computationally intractable — motivating spatial data structures that exploit geometric locality to prune the search space. This project empirically characterises the asymptotic behaviour of **QuadTree** and **KD-Tree**, benchmarked against an exhaustive baseline, with analysis of performance degradation under increasing dimensionality and non-uniform distributions.

2. Algorithms

1.2.1 2.1 Exhaustive KNN (Baseline)

Linear scan over all n points, maintaining a max-heap of size k for the current best neighbours: build $O(1)$, query $O(n \log k)$, space $O(n)$. Highly cache-friendly; provides the baseline against which tree pruning is evaluated.

1.2.2 2.2 QuadTree

Recursively partitions 2D space into four equal quadrants (NW, NE, SW, SE) at each node's spatial midpoint. Built **offline** from a fixed dataset; no rebalancing. Build $O(n \log n)$ avg / $O(n^2)$ worst, query $O(\log n)$ avg, space $O(n)$, fixed at $d = 2$. Subdivision terminates when a node contains $\leq \text{leaf_size}$ points **or** max_depth is reached (required to prevent infinite subdivision on duplicate or clustered points).

1.2.3 2.3 KD-Tree

Recursively partitions m -dimensional space with axis-aligned hyperplanes, cycling through dimensions. Offline build uses **median splits**, guaranteeing balance regardless of distribution. Build $O(n \log n)$, query $O(\log n)$ avg degrading toward $O(n)$ in high dimensions, space $O(n)$, arbitrary d . Pruning skips subtrees whose bounding box is farther than the current best distance — effectiveness degrades with dimensionality (curse of dimensionality), making this a central result of the study.

3. Implementation Plan

All structures and algorithms are implemented in **C++20**; visualisation in **Python 3** with **matplotlib**. All datasets are synthetically generated with a fixed random seed — giving precise distribution control, arbitrary scalability, and no preprocessing overhead.

1.3.1 3.1 Possible Components

Component	Description
<code>exhaustive_knn.cc</code>	Flat array + max-heap KNN
<code>quadtree.cc</code>	2D offline spatial midpoint tree
<code>kdtree.cc</code>	m -d offline median-split tree
<code>data_generator.py</code>	Uniform, Gaussian cluster, skewed distributions
<code>benchmark.cc</code>	Timing harness, CSV output
<code>visualizer.py</code>	Matplotlib animator from C++ logs

1.3.2 3.2 Benchmark Design

Experiments vary: **dataset size** n (primary asymptotic variable); **dimensionality** m (KD-Tree and exhaustive only); **data distribution** (uniform, Gaussian clusters, skewed); **k** (neighbours requested). Each query time is averaged over many independently sampled query points. Metrics: wall-clock query time (mean + std dev), tree build time, and nodes/cells visited per query as a hardware-independent pruning proxy.

1.3.3 3.3 Comparison Matrix

Experiment	Exhaustive	QuadTree	KD-Tree
2D uniform, vary n	✓	✓	✓
2D clustered, vary n	✓	✓	✓
M -D uniform, vary m	✓	—	✓
Vary k	✓	✓	✓

4. Visualisation

A 2D animator will illustrate the search process for all three algorithms via a shared log format emitted by the C++ implementations. Each algorithm's traversal is rendered distinctly: exhaustive shows points scanned with a shrinking best-distance circle; QuadTree shows the cell subdivision with visited/pruned cells highlighted; KD-Tree shows axis-aligned splits with the active bounding box narrowing during descent and backtracking steps visually distinct.

5. Deliverables and Expected Outcomes

By the end of this project we will have implemented and validated all three NNS approaches from scratch in C++, designed and executed a reproducible benchmark suite, and produced a 2D visualisation demonstrating pruning behaviour per algorithm. Results will be analysed against known theoretical complexity, covering the effect of data distribution on tree performance and KD-Tree degradation under increasing dimensionality.

Deliverables: C++ source (exhaustive KNN, QuadTree, KD-Tree, benchmark harness) · Python visualiser with per-algorithm animations · benchmark plots (timing, nodes visited, dimensionality degradation) · written report.

Stack: C++20 · OpenMP (optional) · Python 3 · matplotlib